Corrected

AARHUS UNIVERSITET

Prof. Akhil Arora

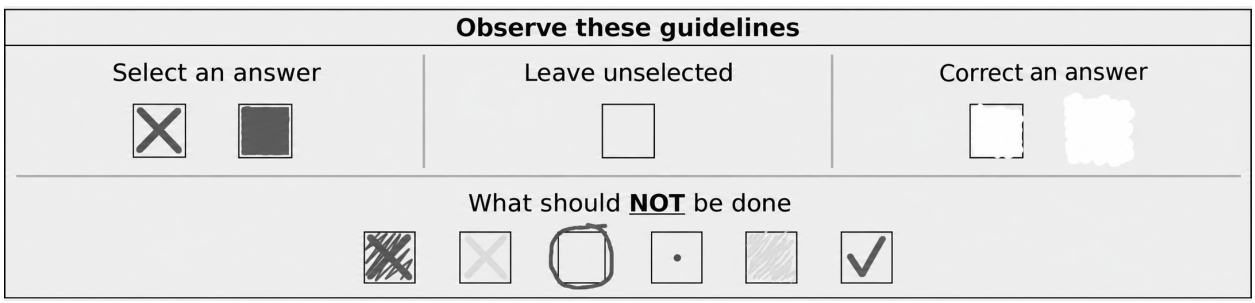
Natural Language Processing (NLP) – 520242U008
Midterm preparation – 2026
Suggested practice time: 42 minutes

Mock Exam 2026 Solution

Student ID : _____ Signature :

Do not turn the page before the start of the exam. This booklet is intended for double-sided printing and has 11 pages, the last ones possibly blank. Do not unstaple.

- This is a closed book exam. Non-programmable calculators are allowed. No other electronic devices of any kind are allowed.
- Place on your desk: your student ID, writing utensils, one double-sided A4 page cheat sheet if you have one; place all other personal items below your desk.
- This mock exam has **14 questions**. The suggested practice time is **42 minutes**.
- Each question has **6 options (A–F)** and is worth **one point**.
- Select **exactly one answer** per question. For statements labeled I, II, III, etc., select the option containing the **complete set of true statements**.
- An **answer sheet is included at the end of this booklet**. Record **all your answers on that sheet**. Only answers marked on the answer sheet will be graded. You can use the blank space in this booklet as scrap paper.
- Use a **black or dark blue ballpen** and clearly erase with **correction fluid** if necessary.
- If a question turns out to be wrong or ambiguous, we may decide to nullify it.
- **Do not tick** the boxes: **cross** them or **fill them in completely**. Do not write close to the boxes, and make sure an erased box is left **completely white**.



Corrected

Question 1

What is the primary purpose of backpropagation in neural networks?

A. To compute gradients of the loss function with respect to the input data.

B. To adjust the learning rate dynamically during training.

C. To directly update the weights while propagating error values backward through the network.

D. To compute gradients of the loss function with respect to the network's parameters.

E. To replace the forward pass when computing the network's predictions.

F. To automatically clear the optimizer's stored gradients after every parameter update.

Answer: D *Difficulty: Easy; Topic: Neural network basics / backpropagation*

Backpropagation applies the chain rule to compute loss gradients with respect to trainable parameters. An optimizer uses those gradients to update the parameters. Input gradients can also be computed, but they are not its primary purpose in training. Backpropagation does not replace the forward pass, choose a learning-rate schedule, or automatically clear stored gradients.

Question 2

A model has perplexity 20 on dataset A and 40 on dataset B, using the same tokenizer and evaluation procedure. Perplexity is the exponential of the average negative log-likelihood per token, using natural logarithms.

I. The average negative log-likelihood per token is lower on A than on B.

II. Every observed token in A must receive exactly twice the probability of every observed token in B.

III. The average negative log-likelihood on B exceeds that on A by $\ln 2$ nats per token.

IV. The perplexity ratio alone does not establish a factor-of-two difference in downstream task performance.

Which option contains exactly the true statements?

A. I only.

B. I and II.

C. II and III.

D. I, II, and III.

E. III and IV.

F. I, III, and IV.

Answer: F *Difficulty: Medium; Topic: Evaluation / perplexity*

I, III, and IV are true. Average negative log-likelihood is the logarithm of perplexity, so the difference is $\ln 40 - \ln 20 = \ln 2$. These aggregate values do not imply a fixed probability ratio for individual tokens, making II false. They also do not determine performance on downstream tasks; differences between the datasets still matter.

Corrected

Question 3

Consider clipped PPO (Proximal Policy Optimization) in an RLHF pipeline.

I. Clipping discourages excessively large changes in action-probability ratios relative to the rollout policy.

II. Clipping guarantees that the KL divergence between the old and new policies never exceeds a fixed bound.

III. Using PPO makes the learned reward model exact and removes its estimation errors.

IV. An RLHF objective can use a separate KL penalty relative to a reference model in addition to PPO clipping.

Which option contains exactly the true statements?

A. I only.

B. I and IV.

C. II and III.

D. I, II, and IV.

E. I, III, and IV.

F. I, II, III, and IV.

Answer: B *Difficulty: Medium; Topic: RLHF / PPO*

I and IV are true. The clipped surrogate objective reduces the incentive for excessively large policy-ratio changes, but does not impose a hard KL-divergence constraint, making II false. A separate reference-model KL penalty is a distinct part of an RLHF objective. Neither mechanism makes the reward model exact, so III is false.

Question 4

Consider standard Word2Vec without subword features.

I. It can produce a distinct learned embedding for a word outside its training vocabulary without any additional mechanism.

II. Standard CBOW and Skip-gram do not explicitly encode the relative positions of words within a fixed context window.

III. Arbitrarily shuffling each sentence leaves all local training co-occurrences, and therefore the learned representations, unchanged.

Which option contains exactly the true statements?

A. II only.

B. I and III.

C. I and II.

D. II and III.

E. I only.

F. I, II, and III.

Answer: A *Difficulty: Hard; Topic: Word embeddings / Word2Vec*

Only II is true. Standard Word2Vec learns embeddings for its training vocabulary and needs another mechanism for a genuinely unseen word. Its usual CBOW and Skip-gram objectives do not encode each context word's relative position. Shuffling an entire sentence can change which words fall within a local window, so III is false. Ignoring order within a fixed window differs from ignoring changes to window membership.

Question 5

Which describes a key limitation of a standard RNN when its current hidden state summarizes a long sequence?

A. Its running time is always quadratic in sequence length, even with a fixed hidden dimension.

B. It must learn a different set of parameters for each timestep.

C. Its hidden-state updates must be linear, so it cannot learn nonlinear representations.

D. It cannot process sequences of different lengths with the same trained parameters.

E. It compresses the preceding context into a fixed-size hidden state, which can make retaining all relevant information difficult.

F. It computes every recurrent state independently, without access to an earlier hidden state.

Answer: E *Difficulty: Easy; Topic: RNN limitations*

A fixed-size current state can create an information bottleneck for long contexts. A standard RNN shares its parameters across timesteps, usually applies a nonlinear state update, and can process different sequence lengths. With a fixed architecture it performs a linear number of recurrent steps; later states depend on earlier states.

Question 6

In a Transformer encoder-decoder model, what is the role of the causal mask in decoder self-attention?

A. It lets the decoder attend to all encoder outputs, aligning source and target sequences.

B. It randomly removes encoder outputs as a regularization mechanism.

C. It prevents a decoder position from attending to future target positions, preserving autoregressive prediction.

D. It removes all earlier target positions, allowing attention only to the current position.

E. It allows future target positions during training and blocks them only during inference.

F. It forces equal attention weights on every available target position.

Answer: C *Difficulty: Easy; Topic: Transformer decoder / masked self-attention*

The causal mask blocks future target positions. With shifted decoder inputs, a prediction cannot access the target token it must predict. Earlier positions remain available, and learned attention scores determine their weights. Cross-attention supplies access to encoder outputs. Causal masking is used during autoregressive training as well as generation.

CORRECTED

Question 7

A training corpus contains the following six sentences. Split words by whitespace; do not add special tokens or form bigrams across sentence boundaries.

```
the cat sat on the mat
the dog chased after the cat
a cat and a dog ran
she sat on the sofa
they bought a new mat
the sofa was comfortable
```

There are 18 unique tokens and 32 tokens in total. Over this fixed vocabulary, use add-one Laplace smoothing:

$$P(A \mid B) = \frac{\text{count}(B, A) + 1}{\text{count}(B) + |V|}.$$

What are $P(\text{sat} \mid \text{on})$ and $P(\text{on} \mid \text{sat})$?

A. $P(\text{sat} \mid \text{on}) = 0$ and $P(\text{on} \mid \text{sat}) = 1$.
B. $P(\text{sat} \mid \text{on}) = \frac{1}{20}$ and $P(\text{on} \mid \text{sat}) = \frac{3}{20}$.
C. $P(\text{sat} \mid \text{on}) = \frac{3}{20}$ and $P(\text{on} \mid \text{sat}) = \frac{1}{20}$.
D. $P(\text{sat} \mid \text{on}) = \frac{1}{18}$ and $P(\text{on} \mid \text{sat}) = \frac{3}{18}$.
E. $P(\text{sat} \mid \text{on}) = \frac{2}{20}$ and $P(\text{on} \mid \text{sat}) = \frac{2}{20}$.
F. $P(\text{sat} \mid \text{on}) = \frac{1}{20}$ and $P(\text{on} \mid \text{sat}) = \frac{2}{20}$.

Answer: B *Difficulty: Hard; Topic: N-gram LM / Laplace smoothing*

The bigram `sat on` occurs twice, while `on sat` never occurs. Both conditioning words occur twice, giving denominator $2 + 18 = 20$. The smoothed numerators are respectively $0 + 1$ and $2 + 1$, so the requested probabilities are 1/20 and 3/20. Smoothing over this vocabulary does not itself solve out-of-vocabulary words.

Question 8

Consider scaled dot-product self-attention, $\text{softmax}(QK^\top/\sqrt{d_k})V$, for a sequence of length n.

I. For each query, softmax normalizes the scores over key positions.
II. Without any mask, self-attention is automatically restricted to preceding positions.
III. Under the usual variance assumptions, division by $\sqrt{d_k}$ helps prevent excessively large scores and softmax saturation.
IV. The score matrix has n^2 entries, and its standard matrix multiplication costs $O(n^2 d_k)$ arithmetic operations.

Which option contains exactly the true statements?

A. I and III.
B. I, II, and III.
C. II and IV.
D. I, II, and IV.
E. I, II, III, and IV.
F. I, III, and IV.

Answer: F *Difficulty: Medium; Topic: Self-attention mechanics*

I, III, and IV are true. Each query compares against the key positions, and softmax normalizes those scores. Under the usual assumptions, dot-product variance grows with d_k, motivating the square-root scaling. Computing all query-key products requires $O(n^2 d_k)$ arithmetic. Causality requires an appropriate mask, so II is false.

Question 9

In a standard BART/T5 model, which representations supply the queries, keys, and values for a decoder block's encoder-decoder cross-attention? Learned projections are applied to these representations.

A. Queries: final encoder outputs; keys and values: decoder hidden states.

B. Queries, keys, and values: the same decoder hidden-state sequence.

C. Queries: decoder hidden states; keys and values: encoder input embeddings, bypassing the encoder layers.

D. Queries: decoder hidden states; keys and values: final encoder outputs.

E. Queries and keys: final encoder outputs; values: decoder hidden states.

F. Queries and keys: decoder hidden states; values: final encoder outputs.

Answer: D *Difficulty: Medium; Topic: Encoder-decoder cross-attention (BART/T5)*

The queries are projections of the decoder representations at that stage of the block. Keys and values are projections of the final encoder outputs, allowing each decoder position to retrieve encoded source information. Using the same decoder sequence for all three describes decoder self-attention. Encoder input embeddings bypass the source encoder; mixing decoder keys with encoder values does not implement the standard cross-attention module.

Question 10

Which statement about these NLP models is correct?

A. T5 can perform generation tasks but cannot classify text by generating a class label.

B. Mean pooling of token embeddings without positional information can distinguish permutations of the same tokens.

C. A standard BERT encoder with its masked-language-model objective is not a left-to-right autoregressive language model.

D. Pretrained ELMo representations alone include a task-specific classification head and a complete sequence-generation system.

E. ELMo always supplies the same context-independent representation for a word, regardless of its surrounding sentence.

F. T5 generates each target token independently, without conditioning on earlier target tokens.

Answer: C *Difficulty: Medium; Topic: Model applicability (T5 / BERT / ELMo)*

BERT's standard bidirectional masked-language-model setup differs from left-to-right autoregressive generation. T5 can express classification as generating a label, and its decoder conditions on earlier target tokens. Mean pooling without positional information is invariant to token permutation. ELMo supplies contextual representations, while downstream tasks require additional components.

Question 11

This function should compute token-level perplexity from a non-empty list containing the probability assigned to each observed next token, with $0 < p_i \leq 1$. Use natural logarithms:

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\ln p_i\right).$$

```
import math

def perplexity(token_probabilities):
    n = len(token_probabilities)
    log_sum = sum(math.log(p) for p in token_probabilities)
    return math.exp(-log_sum)  # BUG: replace this line
```

Which replacement for the marked return line implements the formula for all valid inputs?

A. `return math.exp(-log_sum / n)`

B. `return math.exp(log_sum / n)`

C. `return math.exp(-log_sum) / n`

D. `return -log_sum / n`

E. `return math.exp(-log_sum * n)`

F. `return math.exp(-log_sum / (n + 1))`

Answer: A *Difficulty: Medium; Topic: Code debugging / perplexity*

The negative log probabilities must be averaged before exponentiation. A implements this order. B uses the wrong sign; C divides after exponentiation; D returns average negative log-likelihood rather than perplexity; E multiplies by the token count; F averages using an incorrect count.

Question 12

The classifier below should use the arithmetic mean of all token embeddings in each sequence. Each of the two sequences has five valid tokens, with no padding; the embedding dimension is eight. In `x`, the axes are batch, token position, and embedding coordinate, in that order.
`mean`, `sum`, and `max` reduce the specified dimension. For `max`, `.values` selects the maxima from its result.

```
import torch
from torch import nn

token_ids = torch.tensor([[1, 2, 3, 4, 5],
                          [5, 4, 3, 2, 1]])
embedding = nn.Embedding(10, 8)
classifier = nn.Linear(8, 3)

x = embedding(token_ids)
pooled = __I__
logits = classifier(pooled)
```

Which expression correctly fills `__I__` for any embedding weights?

A. `x.mean(dim=0)`

B. `x.mean(dim=2)`

C. `x.sum(dim=1)`

D. `x[:, 0, :]`

E. `x.mean(dim=1)`

F. `x.max(dim=1).values`

Answer: E *Difficulty: Medium; Topic: Code completion / mean pooling*

Dimension 1 contains the token positions. Averaging it separately for each sequence gives the required arithmetic-mean representation, preserving the batch and embedding axes. A averages across examples; B averages embedding coordinates; C computes a sum without dividing by the number of tokens; D selects only the first token; F performs max pooling.

Corrected

Question 13

The code below should implement causal self-attention. Row i represents a query position and column j a key position. A query may attend to positions $j \leq i$.
Broadcasting constructs a square Boolean mask. `masked_fill` replaces entries where the mask is `True`; softmax over the last dimension normalizes over keys.

```
import torch

length = 4
scores = torch.randn(length, length)
positions = torch.arange(length)

blocked = positions[None, :] < positions[:, None]  # BUG
masked_scores = scores.masked_fill(blocked, float("-inf"))
weights = torch.softmax(masked_scores, dim=-1)
```

I. The buggy condition masks keys at earlier positions, where $j < i$.

II. Replacing `<` with `>` in the marked line masks future keys and leaves the current position available.

III. Changing only the softmax dimension to 0 repairs the causal mask.

IV. Wherever future positions exist, the buggy mask leaves those future keys available to the query.

Which option contains exactly the true statements?

A. I only.

B. I and II.

C. I, II, and IV.

D. II, III, and IV.

E. I, III, and IV.

F. I, II, III, and IV.

Answer: C *Difficulty: Medium; Topic: Code auditing / causal attention masks*

I, II, and IV are true. The left-hand broadcasted tensor contains column indices j and the right-hand tensor contains row indices i. Thus `<` masks the past, while `>` masks the future. The diagonal remains unmasked. Changing the softmax axis does not change which entries were replaced by negative infinity, so III is false.

Corrected

Question 14

This PyTorch loop should make one ordinary optimizer update per minibatch, using only that minibatch's gradients. Gradient accumulation is not intended. The model, optimizer, scalar loss function, and at least two minibatches have already been defined.
`backward()` adds to stored gradients, `zero_grad()` clears them, and `step()` updates parameters without clearing their gradients.

```
model.train()
optimizer.zero_grad()

for inputs, targets in batches:
    logits = model(inputs)
    loss = criterion(logits, targets)
    loss.backward()
    optimizer.step()
```

Which single change corrects the unintended accumulation?

A. Move `optimizer.zero_grad()` inside the loop, immediately after `loss.backward()` and before `optimizer.step()`.

B. Swap `loss.backward()` and `optimizer.step()`.

C. Replace `loss.backward()` with `model.eval()`.

D. Move `optimizer.zero_grad()` to the start of each loop iteration, before the forward pass.

E. Add `model.train()` at the start of every iteration, leaving the existing gradient-reset call outside the loop.

F. Wrap `optimizer.step()` in `torch.no_grad()`, leaving the gradient-reset call outside the loop.

Answer: D *Difficulty: Medium; Topic: Code debugging / minibatch training and gradients*

The original code clears gradients only once. Later backward passes add to the gradients left by previous minibatches. Resetting at the start of each iteration ensures that each update uses only the current minibatch. A discards new gradients before the update; B updates before computing current gradients; C changes model mode instead of computing gradients. E also changes only model mode. F controls gradient tracking during the update but does not clear stored gradients.

Corrected

Mock Exam 2026

First name and last name:

.....

Student ID:

.....

Answer Key

Correct answers are marked.

Answers must be given exclusively on this sheet.
Answers given on the question sheets will be ignored.

Use a black or dark blue ballpen. Cross a box or fill it completely; do not tick. Leave any erased box completely white.

Question 1:	☐ **A**	☐ **B**	☐ **C**	■ **D**	☐ **E**	☐ **F**
Question 2:	☐ **A**	☐ **B**	☐ **C**	☐ **D**	☐ **E**	■ **F**
Question 3:	☐ **A**	■ **B**	☐ **C**	☐ **D**	☐ **E**	☐ **F**
Question 4:	■ **A**	☐ **B**	☐ **C**	☐ **D**	☐ **E**	☐ **F**
Question 5:	☐ **A**	☐ **B**	☐ **C**	☐ **D**	■ **E**	☐ **F**
Question 6:	☐ **A**	☐ **B**	■ **C**	☐ **D**	☐ **E**	☐ **F**
Question 7:	☐ **A**	■ **B**	☐ **C**	☐ **D**	☐ **E**	☐ **F**
Question 8:	☐ **A**	☐ **B**	☐ **C**	☐ **D**	☐ **E**	■ **F**
Question 9:	☐ **A**	☐ **B**	☐ **C**	■ **D**	☐ **E**	☐ **F**
Question 10:	☐ **A**	☐ **B**	■ **C**	☐ **D**	☐ **E**	☐ **F**
Question 11:	■ **A**	☐ **B**	☐ **C**	☐ **D**	☐ **E**	☐ **F**
Question 12:	☐ **A**	☐ **B**	☐ **C**	☐ **D**	■ **E**	☐ **F**
Question 13:	☐ **A**	☐ **B**	■ **C**	☐ **D**	☐ **E**	☐ **F**
Question 14:	☐ **A**	☐ **B**	☐ **C**	■ **D**	☐ **E**	☐ **F**

Total: 14 questions, 14 points.

Before finishing, check that you have transferred your answers to this sheet.